

Aspect	CMS (e.g., WordPress, Drupal, Joomla)	Framework (e.g., Laravel, Django, Ruby on Rails)
Primary Purpose	Manage and publish digital content easily without coding	Build custom web applications from scratch with reusable code components
Target User	Non-technical users (content editors, marketers)	Developers and programmers
Ease of Use	High – user-friendly admin interface	Low – requires coding expertise
Customization Level	Limited to themes, plugins, and existing architecture	Full control over structure, behavior, and design
Out-of-the-Box Features	Comes with built-in content management, user roles, media library, SEO tools	Minimal – provides core libraries, routing, ORM, security, but no ready-to-use frontend
Flexibility	Less flexible – constrained by CMS data model and workflows	Highly flexible – can build any type of web application
Time to Launch	Fast – ready to use after installation	Slow – requires building from ground up
Maintenance Effort	Moderate – core, theme, and plugin updates needed	Higher – developer responsible for security, patches, and upgrades
Learning Curve	Low for users, moderate for developers (templating, hooks)	Steep – must learn MVC, ORM, routing, and architecture patterns
Best For	Blogs, corporate websites, e-commerce (with plugins), news portals	Custom web apps, SaaS products, APIs, enterprise solutions
Examples	WordPress, Drupal, Joomla, Sitecore	Laravel (PHP), Django (Python), Spring (Java), Express (Node.js)

Aspect	JavaScript	TypeScript
Type System	Dynamically typed – variable types are determined at runtime	Statically typed – variable types are defined at compile time
Compilation	Interpreted – runs directly in browsers or Node.js	Compiled – transpiled to JavaScript before execution

Aspect	JavaScript	TypeScript
Type Checking	No type checking – type errors appear at runtime	Yes – type errors are caught during development (compile time)
Learning Curve	Low – beginner-friendly, forgiving syntax	Higher – requires understanding types, interfaces, generics
Tooling & IDE Support	Basic – limited autocompletion and refactoring	Advanced – better autocompletion, navigation, and refactoring
Optional Static Typing	Not available	Yes – types are optional; can be added gradually
Superset / Subset	Standard scripting language	Superset of JavaScript – every JS code is valid TS
Features	Prototypes, closures, event loop, dynamic objects	Types, interfaces, enums, generics, decorators, access modifiers (public/private)
Error Detection	Errors found only at runtime	Errors found at compile time (before execution)
Code Readability	Can become unclear in large codebases due to lack of types	More self-documenting and readable due to explicit types
Adoption & Ecosystem	Universal – runs everywhere; huge ecosystem	Increasing adoption – especially in large-scale enterprise projects
Best For	Small to medium projects, quick prototyping, scripts	Large-scale applications, team projects, complex systems
Example Definition	<code>let name = "John";</code>	<code>let name: string = "John";</code>

Feature	Web 1.0 (1990s–2000)	Web 2.0 (2004–present)	Web 3.0 (emerging)
Read/Write	Read-only	Read-Write	Read-Write-Execute (own)
Content	Static HTML pages	Dynamic, user-generated	Semantic, 3D, decentralized
Interaction	Information retrieval	Social interaction, comments, likes	Trustless interactions, smart contracts
Data ownership	Websites own data	Platforms own data	User owns data (via blockchain)

Feature	Web 1.0 (1990s–2000)	Web 2.0 (2004–present)	Web 3.0 (emerging)
Technologies	HTTP, FTP, HTML/CSS	AJAX, JS frameworks, REST, social APIs	Blockchain, IPFS, RDF, AI agents